Universidad Latina de Costa Rica
Sede Heredia

Sistema de soporte técnico en línea

Manual técnico

Curso: Programación IV
Profesor: Yoltic Wa Pérez Morera
Estudiante: Jose Francisco Viquez Alfaro
Fecha: Noviembre de 2026

Índice

1. Introducción **3**

2. Requisitos y despliegue **3**

2.1. Requisitos 3

2.2. Pasos de instalación 3

2.3. Solución de problemas 4

3. Arquitectura en capas **4**

4. Estructura de paquetes **6**

5. Páginas y endpoints **6**

5.1. Servicio web SOAP 6

6. Modelo de datos **7**

7. Seguridad y manejo de errores **7**

8. Usuarios de prueba **7**

1. Introducción

El sistema permite gestionar tickets de soporte técnico a través de la web. Tiene tres roles (cliente, técnico y administrador), un historial de cada cambio de estado, notificación por correo al cliente cuando cambia el estado de su ticket y un servicio web SOAP para consultar el estado de un ticket.

Estados de un ticket: Abierto → Asignado → En proceso → Resuelto → Cerrado.

Rol	Funciones
Cliente	Crea tickets, consulta el estado y el historial de los suyos y aprueba la solución cuando están *Resuelto* (el ticket pasa a *Cerrado*).
Técnico	Ve solo los tickets que tiene asignados y cambia su estado (Asignado, En proceso, Resuelto).
Administrador	Ve todos los tickets, los asigna a un técnico, gestiona usuarios (crear, editar, eliminar) y consulta el historial general.

Tecnologías

Componente	Tecnología
Lenguaje y compilación	Java 17, Maven (empaquetado WAR)
Presentación	Servlets 3.1, JSP y JSTL 1.2
Acceso a datos	JDBC con consultas parametrizadas, MySQL 8 (utf8mb4)
Servicio web	SOAP con JAX-WS RI 2.3.7 (WSDL generado automáticamente)
Correo	JavaMail (jakarta.mail 1.6.7), Mailtrap como SMTP de pruebas
Servidor	Tomcat 7 embebido mediante plugin de Maven

2. Requisitos y despliegue

2.1. Requisitos

- JDK 17 y Maven 3.6 o superior.
- MySQL.
- Puerto 8080 libre.
- Conexión a Internet la primera vez (Maven descarga las dependencias).

2.2. Pasos de instalación

1. Obtener el proyecto. Se descarga o clona el repositorio (este manual está en su carpeta `docs`) y se abre una terminal en la carpeta principal del proyecto, la que contiene `pom.xml`. Todos los comandos siguientes se ejecutan desde esa carpeta.

2. Crear la base de datos. El script `schema.sql` crea la base `sistema_soporte`, el usuario `soporte_user` (contraseña `123456`), las tablas y los datos de prueba. Ejecutarlo reinicia los datos.

```
# Linux / Mac
```

```
mysql -u root -p < schema.sql
# Linux sin contrasena de root:  sudo mysql < schema.sql
```

En Windows se ejecuta el mismo comando desde CMD, o se abre `schema.sql` en MySQL Workbench y se ejecuta completo.

3. Conexión a la base de datos. Los datos están en `src/main/resources/db.properties` y ya coinciden con el script. Solo se editan si MySQL usa otro puerto o credenciales. También pueden sobrescribirse con las variables de entorno `DB_URL`, `DB_USER` y `DB_PASSWORD`.

4. Correo de notificaciones (opcional). Si no se configura, el sistema funciona igual y solo registra en consola que no pudo enviar el correo.

```
# Linux / Mac
cp src/main/resources/correo.properties.example src/main/resources/correo.properties
# Windows (CMD)
copy src\main\resources\correo.properties.example src\main\resources\correo.properties
```

Después se escriben en `correo.properties` las credenciales SMTP de una bandeja de Mailtrap. Este archivo está en el `.gitignore` y no se sube al repositorio.

5. Ejecutar

```
mvn clean package tomcat7:run
```

Cuando la consola indique que el servidor arrancó, se abre `http://localhost:8080/sistema-tickets/login`. Para detenerlo se usa `Ctrl+C`.

2.3. Solución de problemas

Problema	Solución
`Address already in use`	Otro programa usa el puerto 8080; cerrarlo o detener el Tomcat anterior.
"No se pudo iniciar sesión por un problema del sistema"	MySQL está apagado o `db.properties` no coincide con la base; verificar que `schema.sql` se haya ejecutado.
`Access denied for user 'soporte_user'`	Volver a ejecutar `schema.sql` como `root`, o ajustar `db.properties`.
No llegan correos	Falta `correo.properties` o sus credenciales son incorrectas (la consola muestra `[Correo] ...`); el resto del sistema funciona igual.

3. Arquitectura en capas

El sistema se divide en tres capas, más el paquete de modelo que reúne los objetos que viajan entre ellas.

- **Presentación:** servlets que atienden las peticiones HTTP, validan la sesión y el rol, y entregan la respuesta mediante vistas JSP. Incluye también el servicio web SOAP.
- **Negocio:** servicios que aplican las reglas (quién puede hacer qué, validaciones, transiciones de estado) y envían las notificaciones por correo.

- **Datos:** DAO con JDBC y consultas parametrizadas. Los cambios de estado se hacen en una transacción que incluye el registro en el historial.

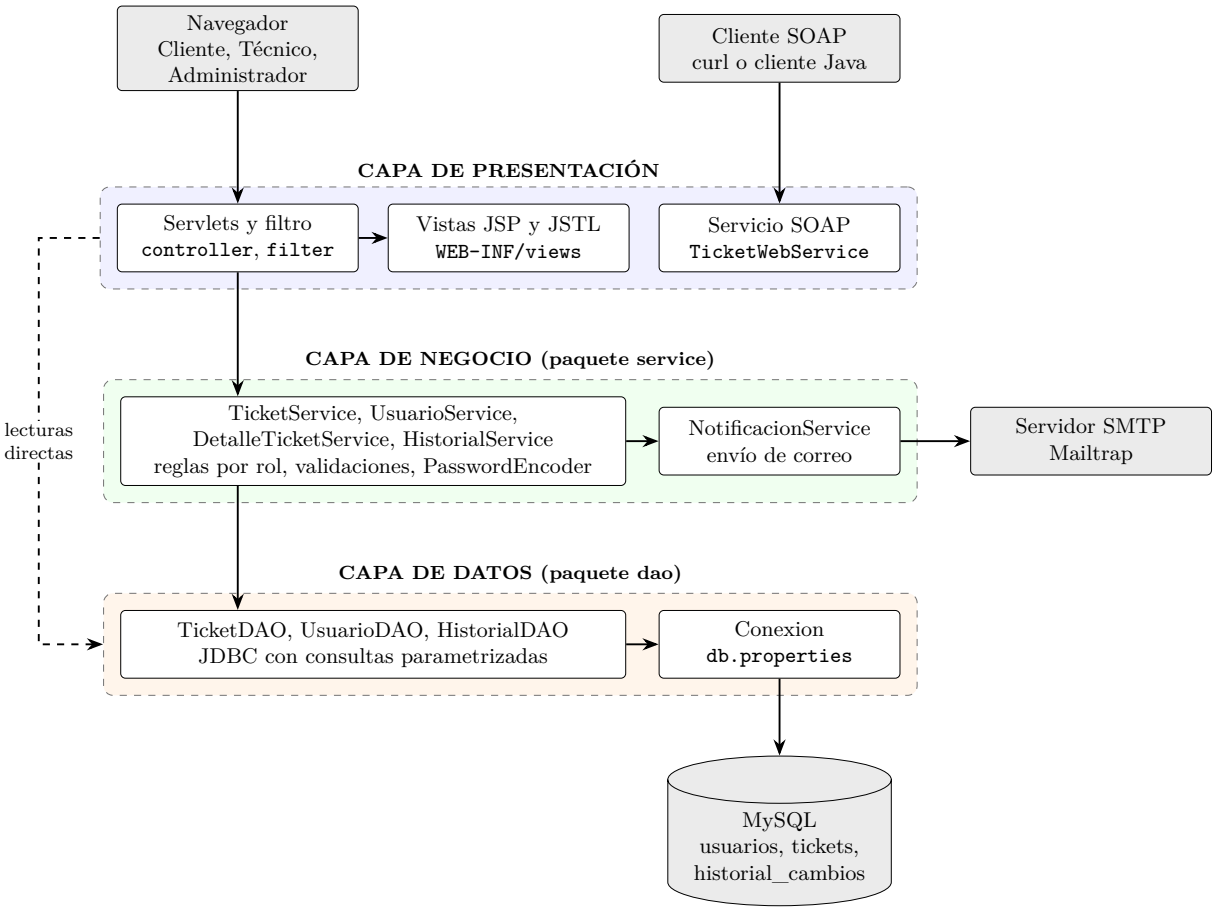


Figura 1: Arquitectura en capas. Las flechas continuas son las llamadas normales entre capas; la flecha punteada indica las consultas de lectura (listados y consulta SOAP) que los servlets y el servicio SOAP hacen directamente a los DAO.

4. Estructura de paquetes

Paquete o carpeta	Contenido
`com.soporte.controller`	Servlets: `LoginServlet`, `LogoutServlet`, `AdminServlet`, `UsuarioServlet`, `HistorialGeneralServlet`, `TecnicoServlet`, `ClienteServlet`, `TicketDetalleServlet`.
`com.soporte.filter`	`CharacterEncodingFilter` (UTF-8 en todas las peticiones).
`com.soporte.service`	Lógica de negocio: `TicketService`, `UsuarioService`, `DetalleTicketService`, `HistorialService`, `NotificacionService`, `PasswordEncoder`.
`com.soporte.dao`	Acceso a datos: `TicketDAO`, `UsuarioDAO`, `HistorialDAO`, `Conexion`.
`com.soporte.model`	Objetos del dominio: `Usuario`, `Ticket`, `HistorialCambio`.
`com.soporte.ws`	Servicio SOAP `TicketWebService` y el cliente de prueba `ClienteSoapPrueba`.
`src/main/resources`	`db.properties` (conexión a MySQL) y `correo.properties.example` (plantilla del correo).
`src/main/webapp`	`login.jsp`, `index.jsp` y `WEB-INF` con `web.xml`, `sun-jaxws.xml` y las vistas JSP por rol (`views/`).
raíz del proyecto	`pom.xml`, `schema.sql`, `README.md`, `.gitignore` y la carpeta `docs` (este manual).

5. Páginas y endpoints

Las vistas JSP están dentro de `WEB-INF/views`, por lo que solo se pueden abrir a través de los servlets y nunca directamente por URL. Todas las rutas cuelgan de `http://localhost:8080/sistema-tickets`.

Ruta	Acceso	Descripción
`/login`	Público	Formulario de inicio de sesión.
`/logout`	Autenticado	Cierra la sesión.
`/cliente/dashboard`	Cliente	Mis tickets, crear ticket y aprobar solución.
`/tecnico/dashboard`	Técnico	Mis tickets asignados y cambio de estado.
`/admin`	Administrador	Todos los tickets y asignación de técnicos.
`/admin/usuarios`	Administrador	Gestión de usuarios.
`/admin/historial`	Administrador	Historial general de cambios (últimos 500).
`/ticket?id=N`	Autenticado	Detalle e historial de un ticket, solo si el rol tiene permiso; si no, responde 404.
`/ws/ticketService`	Público	Servicio SOAP; el WSDL está en `/ws/ticketService?wsdl`.

5.1. Servicio web SOAP

- **WSDL:** `http://localhost:8080/sistema-tickets/ws/ticketService?wsdl`
- **Operación:** `consultarEstadoTicket(ticketId)`, con `ticketId` entero; devuelve el título y el estado del ticket.

- **Espacio de nombres:** `http://ws.soporte.com/`

Ejemplo de consulta con `curl` (se cambia el número del ticket):

```
curl -s -X POST http://localhost:8080/sistema-tickets/ws/ticketService \
  -H "Content-Type: text/xml; charset=utf-8" \
  -d '<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:ws="
    http://ws.soporte.com/"><soapenv:Body><ws:consultarEstadoTicket><ticketId>1</ticketId></ws
    :consultarEstadoTicket></soapenv:Body></soapenv:Envelope>'
```

Respuestas posibles: el estado del ticket (por ejemplo, *El ticket #1 (título) se encuentra en estado: Cerrado*), *Ticket con ID N no encontrado en el sistema* si no existe, y *El ID del ticket no es válido* si el valor no es un entero positivo. También se incluye un cliente de prueba en Java:

```
mvn compile
java -cp target/classes com.soporte.ws.ClienteSoapPrueba 1
```

6. Modelo de datos

Tabla	Descripción
`usuarios`	`id`, `nombre`, `correo` (único), `password_hash`, `rol` (CLIENTE, TECNICO o ADMINISTRADOR).
`tickets`	`id`, `titulo`, `descripcion`, `estado`, `cliente_id`, `tecnico_id` (puede ser nulo) y `fecha_creacion`.
`historial_cambios`	Un registro por cada cambio de estado: ticket, usuario que lo hizo, estado anterior, estado nuevo, comentario y fecha.

7. Seguridad y manejo de errores

- **Autenticación:** las contraseñas se guardan como hash SHA-256 y nunca en texto plano. Al iniciar sesión se crea una sesión nueva y caduca a los 30 minutos.
- **Autorización por rol** en cada servlet: un técnico solo modifica tickets que tiene asignados y un cliente solo ve y aprueba los suyos.
- **Inyección SQL:** todas las consultas JDBC son parametrizadas (`PreparedStatement`).
- **XSS:** la salida de las JSP se escapa con `c:out`.
- **Integridad:** los cambios de estado se ejecutan en una transacción, con bloqueo de fila, que incluye el registro en el historial; si algo falla, se revierte todo.
- **Validación en el servidor:** campos obligatorios y longitudes máximas, estados permitidos, correo único y contraseña de al menos 6 caracteres.
- **Errores:** los fallos de base de datos se informan con un mensaje claro (no como "contraseña incorrecta" ni como una lista vacía), y las páginas de error 400, 404 y 500 no muestran el detalle técnico. Un fallo al enviar el correo no impide el cambio de estado.

8. Usuarios de prueba

Todos los usuarios tienen la contraseña `123456`.

Nombre	Correo	Rol
Admin Principal	admin@soporte.com	ADMINISTRADOR
Técnico Juan	juan.tecnico@soporte.com	TECNICO
Cliente Maria	maria.cliente@gmail.com	CLIENTE
Técnico Carlos	carlos.tecnico@soporte.com	TECNICO
Cliente Lucia	lucia.cliente@gmail.com	CLIENTE